SecureProctor AI — Full Deep Dive (From Zero to Advanced)

Prepared on: 2026-03-16
Project: `proctoring_fastapi`
Purpose: Shareable technical document for explanation, viva, and peer preparation.

---

1) What this system is doing (simple language)

SecureProctor monitors a candidate during interview/exam using webcam frames and decides in real-time whether behavior is **SAFE** or **ALERT**.

It combines:

- Rule-based checks (dark room, no face, multiple faces)
- Landmark-based geometric checks (head pose)
- CNN-based gaze classification (looking at screen vs looking away)
- Temporal smoothing + timeout persistence (to avoid false alarms from single noisy frames)

So this is **not a single-frame classifier**. It is a **stateful proctoring engine**.

---

2) End-to-end architecture

2.1 API layer (control + streaming)

Main API is in `server.py` (FastAPI).

Responsibilities:

- Start/stop proctoring thread
- Return status (`/status`)
- Return settings (`/settings`)
- Serve frame stream (`/video_feed`) and single image (`/frame`)
- Camera management (`/cameras` , `/set_camera/{camera_index}`)
- Session metadata + reports download

2.2 Core engine

Main logic is in `proctor_live.py` .

Responsibilities:

- Open camera with robust backend probing (DSHOW/MSMF/ANY)
- Process each frame
- Decide SAFE/ALERT + reason
- Manage buzzer thread
- Write events, snapshots, video, JSON/CSV report

2.3 Model artifact

- Model class: `EyeCNN` (`eye_cnn_model.py`)
- Runtime weights loaded: `eye_cnn_final.pth`

---

3) Model and learning pipeline (very detailed)

3.1 Label schema

Dataset loader maps folders in this fixed order:

- `good` -> class `0`
- `bad` -> class `1`

This mapping is critical because runtime logic interprets class `1` as suspicious gaze behavior.

3.2 Data preprocessing (train and inference consistency)

Both train and inference use:

- Grayscale image
- Resize to `64x64`
- Normalize pixel values by `/255.0`

This consistency reduces train-test mismatch.

3.3 CNN architecture

`EyeCNN` structure:

1. Conv2D: `1 -> 6`, kernel `5x5`
2. AvgPool2D `(2,2)`
3. Conv2D: `6 -> 16`, kernel `5x5`
4. AvgPool2D `(2,2)`
5. Flatten (`16 * 13 * 13 = 2704`)
6. FC: `2704 -> 120`
7. Dropout `0.3`
8. FC: `120 -> 84`
9. FC: `84 -> 2` (logits)

Parameter count (exact)

- Conv1: `(6*1*5*5) + 6 = 156`
- Conv2: `(16*6*5*5) + 16 = 2,416`
- FC1: `(2704*120) + 120 = 324,600`
- FC2: `(120*84) + 84 = 10,164`
- FC3: `(84*2) + 2 = 170`

Total trainable parameters: [156 + 2,416 + 324,600 + 10,164 + 170 = 337,506]

3.4 Two-stage training strategy (transfer learning style)

Stage-1 pretraining

File: `train_pretrain.py`

- Dataset: `external_dataset`
- Batch size: `32`
- Optimizer: Adam

- Learning rate: `0.001`
- Loss: CrossEntropyLoss
- Epochs: `15`
- Output: `pretrained_eye_cnn.pth`

Stage-2 fine-tuning

File: `train_finetune.py`

- Dataset: `dataset` (local webcam domain)
- Initialize from: `pretrained_eye_cnn.pth`
- Batch size: `16`
- Optimizer: Adam
- Learning rate: `0.0003`
- Loss: CrossEntropyLoss
- Epochs: `10`
- Output: `eye_cnn_final.pth`

Why this helps

- Stage-1 learns general visual patterns (coarse gaze/face cues)
- Stage-2 adapts to your environment (camera angle, lighting, background, device noise)
- Lower LR in stage-2 avoids catastrophic forgetting

---

4) Runtime decision pipeline (frame-by-frame)

For each frame, the engine evaluates checks in this priority order:

1. **Dark environment check**
2. **Face count check** (`NO FACE` , `MULTIPLE FACES`)
3. **Head pose check** (`HEAD LEFT/RIGHT/UP/DOWN`)
4. **CNN gaze check** (`LOOKING AWAY`)

If one earlier check raises a violation in that frame, lower-priority checks are not used for that frame's final reason.

This is a **priority cascade**, not weighted averaging.

---

5) Thresholds, smoothing, and alert trigger math

5.1 Core thresholds

From runtime constants:

- `GAZE_ALERT_TIMEOUT = 0.9s`
- `NO_FACE_TIMEOUT = 0.9s`
- `MULTI_FACE_TIMEOUT = 0.9s`
- `CONF_THRESHOLD = 0.90`
- `DARK_BRIGHTNESS_THRESHOLD = 45`

Dark environment uses shorter timeout (`0.7s`) when triggered.

5.2 Confidence gate (important)

After model inference:

- Compute softmax probabilities
- If `max(prob) < 0.90`, force prediction to class `0` (safe)

Meaning:

- Model only contributes suspicious label when highly confident
- This sacrifices some recall but strongly reduces false positives

5.3 Temporal smoothing buffers

Gaze buffer

- `gaze_buffer` length = 5
- Trigger looking away when `sum(buffer) >= 3`
- Since labels are 0/1, this means at least 3 suspicious predictions in last 5

Head pose buffer

- `head_buffer` length = 5
- Trigger head violation only if same pose appears at least 3 times

This removes instability from small head jitter or tracking flicker.

5.4 Persistence timer (debounce)

Even after a violation candidate appears, alert is not immediate.

Engine tracks:

- `active_violation` key
- `violation_start_time`

Alert condition: [\text{ALERT} \iff (\text{same violation persists}) \land (\Delta t \ge \text{timeout})]

This is why random 1-2 noisy frames do not instantly trigger alarms.

---

6) Head-pose geometry theory

The system uses MediaPipe landmarks:

- Nose tip
- Left eye
- Right eye
- Face top and chin landmarks for normalization

Compute:

- Eye center
- Offset of nose from eye center (x and y)
- Normalize by face height

Ratios:

- Horizontal ratio = `offset_x / face_height`
- Vertical ratio = `offset_y / face_height`

Decision thresholds:

- `> 0.12 => HEAD RIGHT`
- `< -0.12 => HEAD LEFT`
- Vertical dead zone: `-0.10 < ratio < 0.30`
- `>= 0.30 => HEAD DOWN`
- `<= -0.30 => HEAD UP`

Normalization by face height makes thresholds more scale-invariant across distance changes.

---

7) SAFE/ALERT state machine behavior

SAFE path

If no persistent violation:

- `status = SAFE`
- `reason = SAFE`
- buzzer stopped
- violation timers/buffers reset as needed

ALERT path

If violation persists longer than threshold:

- `status = ALERT`
- reason set (e.g., `LOOKING AWAY`, `NO FACE`, etc.)
- buzzer starts

Event transitions

On status/reason change:

- Record event (`ALERT_TRIGGERED` or `STATUS_RECOVERED`)
- Capture snapshot for alert events
- Keep timeline for report generation

---

8) Reporting and auditability

When session starts/stops, engine records:

- Session start/end timestamps
- Event timeline with offsets
- Alert reason counts
- Detection settings used
- Snapshot files
- Video recording (if initialized)

Reports generated in:

- JSON summary
- CSV event log
- MP4 session video
- Alert snapshots

- ZIP package download endpoint

This provides explainability and post-exam audit trail.

---

9) Camera robustness and fallback strategy

Camera opening strategy

- Try selected camera index if provided
- Else probe external indexes first
- Else fallback to index 0
- Use multiple backends: DSHOW, MSMF, ANY

Detector fallback

- Primary: MediaPipe FaceMesh (landmarks + better face tracking)
- Fallback: OpenCV Haar face detector if MediaPipe unavailable

In fallback mode:

- Face count + CNN gaze still work
- Landmark-based head pose is unavailable

---

10) Statistical intuition: why this is stable

Suppose per-frame false suspicious probability is `p`.

With 5-frame majority trigger (`>=3`), chance of accidental trigger in a random window is: [P = \sum_{k=3}^{5} \binom{5}{k} p^k(1-p)^{5-k}]

Examples:

- If `p = 0.10`, `P ≈ 0.00856`
- If `p = 0.20`, `P ≈ 0.05792`

Then timeout persistence (`~0.9s`) further suppresses accidental alerts.

So system stability comes from **confidence gate + temporal vote + persistence timer**.

---

11) Known limitations (honest technical points)

1. Training scripts log loss only; no integrated validation metrics (accuracy/F1/AUC) in current scripts.
2. Raw dataset folders (`dataset`, `external_dataset`) are not present in this snapshot, so exact sample counts are not recoverable here.
3. Confidence threshold `0.90` appears engineering-tuned, not formally calibrated via ROC/PR sweep in code.
4. Buzzer uses Windows `winsound`, so cross-platform audio handling is limited.
5. Binary gaze labeling (`good/bad`) may not represent full real-world gaze complexity.

---

12) Improvement roadmap (advanced suggestions)

1. Add train/val/test split with stratification and metric logging.
2. Add confusion matrix + precision/recall/F1 per run.
3. Calibrate threshold using validation ROC/PR and choose operating point by target false alarm rate.
4. Add augmentation (brightness, blur, slight rotation, occlusion) for better robustness.

5. Replace binary gaze with multi-class gaze zones + uncertainty estimation.
6. Add per-user calibration phase at session start.
7. Fuse face mesh confidence, eye aspect ratio, and CNN logits in a probabilistic framework.

---

13) Viva-ready explanation script (short)

"Our proctoring engine is a cascaded and stateful decision system. At frame level, we first apply hard constraints (dark room, no face, multiple faces), then geometric head-pose checks from facial landmarks, and finally CNN-based gaze classification. We do not raise alerts from one frame. Instead, we use confidence gating (0.90), temporal majority voting over recent frames, and persistence timers (~0.9 seconds). This combination reduces false positives while preserving detection sensitivity. The model itself is trained in two stages: pretraining on external data and fine-tuning on local webcam data to handle domain shift. Every alert transition is logged with timestamp, snapshot, and report artifacts for auditability."

---

14) Viva cross-questions and strong answers

Q1) Why not trigger alert on single bad frame?

Because webcam noise, motion blur, and transient pose jitter create false positives. Temporal smoothing and timeout persistence convert noisy frame-level predictions into reliable behavioral decisions.

Q2) Why confidence threshold = 0.90?

To avoid uncertain predictions causing alerts. It biases toward precision. This is safer in exam context where false accusation is costly.

Q3) Why two-stage training?

Pretraining learns generic visual features. Fine-tuning adapts to target camera/lighting/domain. This improves generalization over training only on small local data.

Q4) Why class mapping matters?

Folder order defines numeric labels. If mapping flips, runtime interpretation of suspicious class flips and system behavior becomes incorrect.

Q5) What is the difference between rule-based and learned components here?

Rule-based checks are deterministic constraints (darkness, face count). Learned component (CNN) handles subtle gaze patterns not expressible by simple rules.

Q6) Why use normalized geometric ratios for head pose?

Normalization by face height makes thresholds less sensitive to distance from camera and frame scale changes.

---

15) Quick one-page summary (for your friend)

- **Model**: EyeCNN (337,506 params), binary classes: good(0)/bad(1)
- **Training**: Pretrain (`lr=1e-3` , `bs=32` , `15 epochs`) -> Fine-tune (`lr=3e-4` , `bs=16` , `10 epochs`)
- **Inference gate**: softmax confidence must be >= 0.90, else forced safe
- **Smoothing**: gaze majority vote (3 of 5), head-pose consistency (>=3)
- **Timeouts**: mostly 0.9s; dark environment 0.7s

- **Alert logic**: persistent violation only -> ALERT + buzzer + snapshot
- **Reports**: JSON + CSV + MP4 + snapshots + ZIP
- **Design goal**: high reliability with low false alarms

---

16) Notes for sharing

- This document reflects current implementation in:
  - `proctor_live.py`
  - `eye_cnn_model.py`
  - `train_pretrain.py`
  - `train_finetune.py`
  - `dataset_loader.py`
  - `server.py`

If needed, create a second version with screenshots/diagrams for non-technical audience.